

Day 30 — Week 6 Project Brief

Tree-Based Fraud Detection Pipeline

Phase 2 · Week 6 · Day 30 of 120

Estimated focused time: 6–7 hours

Dataset: insurance_claims.csv (8,000 claims, 3.7% fraud)

Tools: pandas, scikit-learn, xgboost, imbalanced-learn, shap

Read this first — what this project is actually about

The point of this project is not the highest score. It is to make a series of genuine decisions and be able to defend every one of them. A pipeline with a PR-AUC of 0.12 and clear reasoning is better work than one with 0.16 and none, because only the first tells you why it behaves the way it does, or what to change when it stops working.

Every choice you make must be accompanied by the alternative you rejected and the evidence that decided between them. A choice with no stated alternative is not a decision — it is a default you inherited without noticing.

To make the standard concrete, here are two submissions describing the same step:

Weak — this tells a reader nothing:

"I used SMOTE to handle the class imbalance, as it is the standard approach for imbalanced datasets."

Strong — this is the standard to aim for:

"I compared SMOTE against class weights and a do-nothing baseline. SMOTE gave the best ranking (PR-AUC 0.149 vs 0.110 for class weights), but class weights caught eleven times more fraud at the default threshold (recall 0.297 vs 0.027). Because the investigation team needs cases surfaced rather than ranked, and because I tune the threshold separately in step 5, I chose class weights. The cost is a doubled Brier score (0.071 vs 0.035), so I calibrate in step 6 before any probability is used numerically."

The difference is not length. It is that the second one names the alternatives, cites the numbers, connects the choice to the purpose, and acknowledges what it cost.

The scenario

You are a data scientist at an insurance company. The claims team currently reviews flagged claims manually and can investigate roughly 200 claims per month. They have asked you to build a model that identifies which claims are worth investigating.

What the business told you, in their words:

- "A fraudulent claim we pay out costs us roughly twenty times what it costs to investigate a legitimate one."
- "We can investigate about 200 claims a month. More than that and the queue backs up."
- "We want to know why a claim was flagged. Our investigators will not act on a black box."
- "Eventually we want to multiply your risk score by the claim amount to prioritise the queue."

Each of those four sentences constrains a technical decision you will make. Part of your job is working out which one constrains what — the brief does not tell you.

What you deliver

- A repository containing your notebook, the dataset, and a README.md.
- The notebook must run top to bottom on a fresh kernel without errors.
- A decision log (described below) — the most important part of the submission.
- A one-page recommendation written for the claims team, not for a data scientist.

The decision log — the core of this project

Maintain a table in your README with one row per significant decision. Every row must have all five columns filled:

Decision	Options considered	Evidence	Choice	What it cost me
Imbalance handling	none / class weights / SMOTE	PR-AUC 0.142/0.110/0.149, recall 0.027/0.297/0.027	class weights	Brier doubled, so I calibrated after

A minimum of eight rows. If a decision has no cost, you have probably not thought about it hard enough — almost every modelling choice trades one thing for another.

Required work

STEP 1 — Understand the problem before modelling

- Report the fraud rate and imbalance ratio.
- Compute the accuracy of a model that predicts 'legitimate' for every claim. State plainly why this makes accuracy an unusable metric here.
- Choose your evaluation metrics and justify each one. State what each measures and why it suits THIS problem.
- Report the random baseline for PR-AUC and explain what it is.

You must be able to answer

- Why is ROC-AUC flattering on this dataset?
- If your PR-AUC is 0.14, is that good? Compared to what?

STEP 2 — Explore with a purpose

Exploration is not decoration. Every chart you include must answer a question you can state.

- Investigate how fraud rate varies across at least four features.
- Identify at least one feature that appears predictive and one that appears useless. Show the evidence for both.
- Handle the missing values in age and days_to_report — and justify your method rather than defaulting to the median.
- For each chart, write one sentence stating the question it answers and what you concluded.

Trap to avoid

Do not produce twenty plots and no conclusions. Five plots with five conclusions beats twenty plots with none. Any chart you cannot write a conclusion for should be deleted.

STEP 3 — Compare three model families

- Train a Decision Tree, a Random Forest, and XGBoost on identical splits.

- Report your chosen metrics for all three in one table.
- Report each model's train-test gap and state which is overfitting most.
- Choose one model to carry forward and justify it — including why you rejected the other two.

You must be able to answer

- If the Decision Tree scores close to XGBoost, what would justify using the simpler model?
- Did you compare them fairly — same split, same features, same preprocessing?

STEP 4 — Handle the imbalance deliberately

- Test at least three approaches: no handling, class weights, and one resampling method.
- Report PR-AUC, recall, precision AND Brier score for each.
- Choose one and justify it with reference to what the model is for.
- If your chosen method distorted the probabilities, say so explicitly and state how you will address it.

Trap to avoid

SMOTE must be applied to the training data only, inside the cross-validation loop if you use one. Applying it before the split leaks synthetic positives into your test set and invalidates every number that follows.

STEP 5 — Choose a threshold from costs, not habit

- State your false positive and false negative costs explicitly, using the business statement above.
- Plot total cost against threshold across the full range.
- Report the cost-optimal threshold and how much it improves on the default 0.50.
- Check your chosen threshold against the capacity constraint: does it flag more than 200 claims a month? If so, reconcile the two.

You must be able to answer

- If the cost ratio were 5:1 instead of 20:1, would your threshold move up or down? Verify it.
- Which of the two constraints — cost or capacity — is binding for your model, and what would you tell the business about that?

STEP 6 — Decide about calibration

- Measure calibration before and after, using Brier score and a calibration curve.
- If you resampled or weighted, calibrate on data with the REAL class balance, not the resampled one, and explain why that matters.
- Decide whether your use case needs calibrated probabilities and justify the decision either way.

You must be able to answer

- The business wants to multiply your score by the claim amount. Does that require calibration? Why?
- Can a model rank perfectly and still be badly calibrated? Demonstrate or explain.

STEP 7 — Interpret the model

- Produce a SHAP summary plot and identify the top drivers with their direction.
- Produce SHAP explanations for three individual claims: one confident fraud, one confident legitimate, one uncertain.
- Sanity-check the top drivers against domain intuition. Does the model agree with what a fraud investigator would expect?
- Check correlations between your features before reading SHAP importance, and state what you found.

You must be able to answer

- Is there any feature whose importance you find suspicious? What would you investigate?

- If the model relied heavily on a feature that would not be available at prediction time in production, what would that mean?

STEP 8 — Write the recommendation

One page, addressed to the claims team. They are not data scientists.

- What the model does and how well it works, in business terms rather than metrics.
- The threshold you recommend and what it means for their workload.
- What drives the predictions, in plain language.
- Two honest limitations.
- What you would do next with another week.

Mistakes that invalidate your results

These are not style issues. Each one makes the numbers in your notebook untrue, which means every conclusion built on them is untrue as well. Check for all of them before you submit.

- Applying SMOTE or any resampling before the train/test split. Synthetic positives leak into your test set, and every score after that point is inflated and meaningless.
- Calibrating on resampled data. You will produce probabilities calibrated to a 50% fraud world that does not exist.
- Tuning anything against the test set. Once you have used it to make a choice it is no longer an honest estimate of future performance — use a validation split instead.
- Reporting accuracy as a headline number on a 3.7% problem. A do-nothing model scores 96%.
- Using threshold 0.5 without discussing it. On this data it catches 2 frauds out of 74.
- Imputing missing values using statistics computed on the full dataset rather than the training split alone. This is a subtle leak and easy to do by accident.

Self-check before you submit

Answer each of these honestly. Any one you cannot answer is a gap worth fixing before you submit.

- Can I name the alternative I rejected for every significant choice I made?
- Is there any number in my notebook I cannot explain the origin of?
- Did my test set influence any decision before the final evaluation?
- Would my recommendation make sense to someone who has never heard of PR-AUC?
- Have I stated at least two genuine limitations, rather than token ones?
- If my model were deployed tomorrow and performed badly, would my notebook help someone work out why?

Timeline

- Hour 1: compressed lecture notebook (Day30_Imbalance_Threshold_Calibration.ipynb), worked through.
- Hours 2–3: steps 1–3.
- Hours 4–5: steps 4–6, the decision-heavy part. Budget the most time here.
- Hour 6: step 7, interpretation.
- Hour 7: step 8, decision log, README, final clean run.

CodeTrade AI/ML Internship · Day 30 Project Brief · Tree-Based Fraud Detection Pipeline